

Laboratorio 8:

Ángel Mérida 23661

Problema 1: 25%

Analice el siguiente programa

```
1 void function(int n) {  
2     int i, j, k, counter = 0;  
3     for (i = n/2; i <= n; i++) {  
4         for (j = 1; j+n/2 <= n; j++) {  
5             for (k = 1; k <= n; k = k*2) {  
6                 counter++;  
7             }  
8         }  
9     }  
10 }
```

Parte a

Encuentre la complejidad de tiempo en notación Big-Oh. Muestre todo su procedimiento.

Parte a: complejidad

Paso 1. Los ciclos son independientes. Los límites de los tres ciclos dependen solo de n , nunca de i , j o k . Por eso el total es el producto de las vueltas de cada ciclo:

$$T(n) = \sum_i \sum_j \sum_k 1 = (\text{vueltas de } i) \cdot (\text{vueltas de } j) \cdot (\text{vueltas de } k)$$

Paso 2. Ciclo i (línea 3). i toma los valores $\lfloor n/2 \rfloor, \lfloor n/2 \rfloor + 1, \dots, n$:

$$\text{vueltas de } i = n - \lfloor n/2 \rfloor + 1 = \lfloor n/2 \rfloor + 1$$

Paso 3. Ciclo j (línea 4). La condición $j + \lfloor n/2 \rfloor \leq n$ equivale a $j \leq n - \lfloor n/2 \rfloor = \lfloor n/2 \rfloor$, y j empieza en 1:

$$\text{vueltas de } j = \lfloor n/2 \rfloor$$

Paso 4. Ciclo k (línea 5). k no suma, se duplica: toma 1, 2, 4, ..., 2^m . Sigue mientras $2^m \leq n$, o sea $m \leq \log n$, así que el último exponente es $m = \lfloor \log n \rfloor$ y los exponentes van de 0 a $\lfloor \log n \rfloor$:

$$\text{vueltas de } k = \lfloor \log n \rfloor + 1$$

Paso 5. Total exacto.

$$T(n) = (\lfloor n/2 \rfloor + 1) \cdot \lfloor n/2 \rfloor \cdot (\lfloor \log n \rfloor + 1)$$

Ejemplo: $n = 10 \rightarrow (5 + 1) \cdot 5 \cdot (3 + 1) = 120$, que es justo el valor final de counter que da el programa.

Paso 6. Cotas. Para $n \geq 2$ se cumple $\lceil n/2 \rceil \leq n$, $\lceil n/2 \rceil + 1 \leq n$ y $\lfloor \log n \rfloor + 1 \leq 2 \log n$; y siempre $\lceil n/2 \rceil \geq n/2$ y $\lfloor \log n \rfloor + 1 \geq \log n$. Entonces:

$$\frac{1}{4} \cdot n^2 \log n \leq T(n) \leq 2 \cdot n^2 \log n \text{ para todo } n \geq 2$$

La cota superior da $T(n) = O(n^2 \log n)$ con $c = 2$ y $n_0 = 2$, y la inferior da $\Omega(n^2 \log n)$ con $c = \frac{1}{4}$. Por lo tanto:

$$T(n) = \Theta(n^2 \log n) \Rightarrow O(n^2 \log n)$$

Problema 2: 25%

Analice el siguiente programa:

```
1 void function(int n) {  
2     if (n <= 1) return;  
3     int i, j;  
4     for (i = 1; i <= n; i++) {  
5         for (j = 1; j <= n; j++) {  
6             printf("Sequence\n");  
7             break;  
8         }  
9     }  
10 }
```

Parte a

Encuentre la complejidad de tiempo en notación Big-Oh. Muestre todo su procedimiento.

Parte a: complejidad

Paso 1. Línea 2. Si $n \leq 1$ la función termina de inmediato: costo constante y ningún printf.

Paso 2. Ciclo i (línea 4). i va de 1 a n: n vueltas.

Paso 3. Ciclo j (línea 5): la clave es el break. En cada vuelta de i, j empieza en 1, la condición $1 \leq n$ es verdadera ($n \geq 2$), se ejecuta el printf y el break de la línea 7 sale del ciclo j. El incremento $j++$ nunca se ejecuta. El ciclo j da exactamente 1 vuelta, no n:

$$T(n) = \sum_{i=1}^n 1 = n \quad (n \geq 2), \quad T(n) = 0 \quad (n \leq 1)$$

Contando también las comparaciones de los ciclos: la condición de i se evalúa $n + 1$ veces y la de j n veces, así que el total de pasos elementales es de la forma $a \cdot n + b$, también lineal.

Paso 4. Cotas. Para $n \geq 2$, $1 \cdot n \leq T(n) \leq 1 \cdot n$, así que $T(n) = O(n)$ y $T(n) = \Omega(n)$ con $c = 1$ y $n_0 = 2$:

$$T(n) = \Theta(n) \Rightarrow O(n)$$

Problema 3: 50%

Analice el siguiente programa:

```
1 void function(int n) {  
2     int i, j;  
3     for (i = 1; i <= n/3; i++) {  
4         for (j = 1; j <= n; j += 4) {  
5             printf("Sequence\n");  
6         }  
7     }  
8 }
```

Parte a

Encuentre la complejidad de tiempo en notación Big-Oh. Muestre todo su procedimiento.

Parte a: complejidad

Paso 1. Independencia. Los límites de i y de j dependen solo de n , así que el total es el producto de las vueltas:

$$T(n) = \sum_{i=1}^{\lfloor n/3 \rfloor} \sum_{t=0}^{\lfloor (n-1)/4 \rfloor} 1 \text{ (con } j = 1 + 4t)$$

Paso 2. Ciclo i (línea 3). i va de 1 a $\lfloor n/3 \rfloor$ (división entera):

$$\text{vueltas de } i = \lfloor n/3 \rfloor$$

Paso 3. Ciclo j (línea 4). j avanza de 4 en 4: $j = 1 + 4t$ con $t = 0, 1, 2, \dots$. La condición $1 + 4t \leq n$ da $t \leq (n-1)/4$, o sea $t = 0, \dots, \lfloor (n-1)/4 \rfloor$:

$$\text{vueltas de } j = \lfloor (n-1)/4 \rfloor + 1 = \lfloor n/4 \rfloor$$

Paso 4. Total exacto.

$$T(n) = \lfloor n/3 \rfloor \cdot \lfloor n/4 \rfloor \approx n^2 / 12$$

Ejemplos: $n = 10 \rightarrow 3 \cdot 3 = 9$; $n = 100 \rightarrow 33 \cdot 25 = 825$, iguales a las impresiones que cuenta verificar.py.

Paso 5. Cota superior. $\lfloor n/3 \rfloor \leq n/3$ y $\lfloor n/4 \rfloor \leq n/4 + 1 \leq n/2$ para $n \geq 4$. Entonces:

$$T(n) \leq (n/3)(n/2) = n^2/6 \text{ para } n \geq 4 \Rightarrow T(n) = O(n^2) \text{ (c = 1/6, } n_0 = 4)$$

Paso 6. Cota inferior. $\lfloor n/3 \rfloor \geq n/3 - 1 \geq n/6$ para $n \geq 6$, y $\lfloor n/4 \rfloor \geq n/4$. Entonces:

$$T(n) \geq (n/6)(n/4) = n^2/24 \text{ para } n \geq 6 \Rightarrow T(n) = \Omega(n^2) \text{ (c = 1/24, } n_0 = 6)$$

Paso 7. Conclusión.

$$T(n) = \Theta(n^2) \Rightarrow O(n^2)$$

Problema 4: 25%

Encuentre el **mejor caso**, **caso promedio** y **peor caso** del algoritmo de Búsqueda Lineal (Linear Search, Binary Search y Quick sort). Muestre todo su procedimiento.

Búsqueda lineal

Recorre $A[1], A[2], \dots$ comparando cada elemento con x hasta encontrarlo o terminar el arreglo.

Mejor caso. x está en $A[1]$: se hace 1 comparación. $T(n) = 1 = \Theta(1)$.

Peor caso. x está en $A[n]$ o no está: se compara con los n elementos. $T(n) = n = \Theta(n)$.

Caso promedio. Suponiendo que x está en el arreglo y que cada posición es igual de probable (probabilidad $1/n$), encontrarlo en la posición i cuesta i comparaciones:

$$E[T(n)] = \sum_{i=1}^n i \cdot (1/n) = (1/n) \cdot n(n+1)/2 = (n+1)/2 = \Theta(n)$$

Si además x puede no estar (con probabilidad $1-p$), $E[T(n)] = p(n+1)/2 + (1-p)n$, que sigue siendo $\Theta(n)$. En promedio se revisa la mitad del arreglo: mitad de trabajo, mismo orden.

Búsqueda binaria (arreglo ordenado)

Compara x con el elemento del medio; si no es, descarta la mitad donde no puede estar y repite. Cada vuelta cuenta como una comparación (de tres vías: igual, menor o mayor).

Mejor caso. x es justo el elemento del medio: 1 comparación. $T(n) = \Theta(1)$.

Peor caso. x no está o se encuentra al final de la búsqueda. Cada vuelta deja a lo sumo la mitad de los elementos:

$$T(n) = T(\lfloor n/2 \rfloor) + 1, T(1) = 1$$

Desenrollando: $T(n) = T(n/2) + 1 = T(n/4) + 2 = \dots = T(n/2^k) + k$. Se llega a un solo elemento cuando $n/2^k = 1$, es decir $k = \log n$:

$$T(n) = \log n + 1 \text{ (exactamente } \lfloor \log n \rfloor + 1) \Rightarrow \Theta(\log n)$$

Con el Teorema Maestro: $a = 1$, $b = 2$, $f(n) = 1$ y $n^{\log_b a} = n^0 = 1 = \Theta(f(n))$, caso 2, así que $T(n) = \Theta(\log n)$.

Caso promedio. Las comparaciones forman un árbol de decisión. Si $n = 2^k - 1$ el árbol está completo: en el nivel d ($d = 1, \dots, k$) hay 2^{d-1} elementos y encontrar cualquiera de ellos cuesta d comparaciones. Con x presente y todas las posiciones igual de probables:

$$E[T(n)] = (1/n) \sum_{d=1}^k d \cdot 2^{d-1} = ((k-1)2^k + 1) / n = k - 1 + k/n$$

usando la identidad $\sum_{d=1}^k d \cdot 2^{d-1} = (k-1)2^k + 1$ y $n+1 = 2^k$. Como $k = \log(n+1)$:

$$E[T(n)] \approx \log(n+1) - 1 \Rightarrow \Theta(\log n)$$

Ejemplo: $n = 15$ ($k = 4$) $\rightarrow (3 \cdot 16 + 1)/15 = 3.27$ comparaciones en promedio, contra 4 en el peor caso. Más de la mitad de los elementos están en el último nivel, por eso el promedio queda a una comparación del peor caso.

Quicksort (partición de Lomuto, pivot = último elemento)

Particionar m elementos compara cada uno de los otros $m-1$ contra el pivot: $m-1$ comparaciones. Luego se ordenan recursivamente las dos partes. Si el pivot queda en la posición q :

$$T(n) = T(q-1) + T(n-q) + (n-1), T(0) = T(1) = 0$$

Mejor caso. El pivot siempre queda en medio y parte el arreglo en dos mitades:

$$T(n) = 2T(n/2) + (n-1)$$

En el árbol de recursión cada nivel suma a lo más n comparaciones y hay $\log n$ niveles (el tamaño se divide entre 2 cada vez), así que $T(n) \approx n \log n$. Con el Teorema Maestro: $a = 2$, $b = 2$, $n^{\log_2 2} = n$ y $f(n) = \Theta(n)$, caso 2: **$T(n) = \Theta(n \log n)$** .

Peor caso. El pivot siempre es el mínimo o el máximo, por ejemplo un arreglo ya ordenado con pivot = último. Una parte queda vacía y la otra con $n-1$ elementos:

$$T(n) = T(n-1) + (n-1) = (n-1) + (n-2) + \dots + 1 = n(n-1)/2 \Rightarrow \Theta(n^2)$$

Caso promedio. Con una permutación aleatoria, el pivot tiene la misma probabilidad $1/n$ de quedar en cualquier posición q :

$$T(n) = (n-1) + (1/n) \sum_{q=1}^n [T(q-1) + T(n-q)] = (n-1) + (2/n) \sum_{k=0}^{n-1} T(k)$$

(las dos sumas recorren los mismos valores $T(0), \dots, T(n-1)$). Se multiplica por n , se escribe la misma ecuación para $n-1$ y se restan para eliminar la sumatoria:

$$nT(n) - (n-1)T(n-1) = 2(n-1) + 2T(n-1) \Rightarrow nT(n) = (n+1)T(n-1) + 2(n-1)$$

Dividiendo entre $n(n+1)$ y llamando $S(n) = T(n)/(n+1)$:

$$S(n) = S(n-1) + 2(n-1)/(n(n+1)) = S(n-1) + 4/(n+1) - 2/n$$

Sumando desde 2 hasta n con $S(1) = 0$ y $H_n = 1 + 1/2 + \dots + 1/n$ (número armónico):

$$S(n) = 2H_n + 4/(n+1) - 4 \Rightarrow T(n) = 2(n+1)H_n - 4n$$

Como $H_n \approx \ln n$, $T(n) \approx 2n \ln n \approx 1.39 n \log n$. **$T(n) = \Theta(n \log n)$** : el caso promedio solo es ~39 % más caro que el mejor caso.

Resumen:

Algoritmo	Mejor caso	Caso promedio	Peor caso
Búsqueda lineal	$\Theta(1)$	$(n+1)/2 \rightarrow \Theta(n)$	$n \rightarrow \Theta(n)$
Búsqueda binaria	$\Theta(1)$	$\approx \log(n+1) - 1 \rightarrow \Theta(\log n)$	$\lfloor \log n \rfloor + 1 \rightarrow \Theta(\log n)$
Quicksort	$\approx n \log n \rightarrow \Theta(n \log n)$	$2(n+1)H_n - 4n \rightarrow \Theta(n \log n)$	$n(n-1)/2 \rightarrow \Theta(n^2)$

Problema 5

Definiciones usadas (para funciones no negativas, como los tiempos de ejecución): $f = O(g)$ si existen $c > 0$ y n_0 con $f(n) \leq c \cdot g(n)$ para $n \geq n_0$; $f = \Omega(g)$ si existen $c > 0$ y n_0 con $f(n) \geq c \cdot g(n)$ para $n \geq n_0$; $f = \Theta(g)$ si $f = O(g)$ y $f = \Omega(g)$.

a) Si $f = \Theta(g)$ y $g = \Theta(h)$, entonces $h = \Theta(f)$. **VERDADERO**

Por hipótesis existen constantes positivas tales que:

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ para } n \geq n_1, \quad d_1 h(n) \leq g(n) \leq d_2 h(n) \text{ para } n \geq n_2$$

Transitividad. Para $n \geq \max(n_1, n_2)$ se sustituye la segunda en la primera:

$$c_1 d_1 h(n) \leq f(n) \leq c_2 d_2 h(n)$$

Simetría. Despejando h de cada desigualdad:

$$(1 / (c_2 d_2)) \cdot f(n) \leq h(n) \leq (1 / (c_1 d_1)) \cdot f(n)$$

Las dos constantes son positivas, así que $h = \Theta(f)$ con $n_0 = \max(n_1, n_2)$. En otras palabras, Θ es transitiva y simétrica (es una relación de equivalencia).

b) Si $f = O(g)$ y $g = O(h)$, entonces $h = \Omega(f)$. **VERDADERO**

Por hipótesis $f(n) \leq c_1 g(n)$ para $n \geq n_1$ y $g(n) \leq c_2 h(n)$ para $n \geq n_2$. Para $n \geq \max(n_1, n_2)$:

$$f(n) \leq c_1 g(n) \leq c_1 c_2 h(n) \Rightarrow f = O(h) \text{ (transitividad de } O)$$

Despejando h :

$$h(n) \geq (1 / (c_1 c_2)) \cdot f(n)$$

que es exactamente la definición de $h = \Omega(f)$ con $c = 1/(c_1 c_2) > 0$. Se usó que $f = O(h)$ si y solo si $h = \Omega(f)$ (simetría transpuesta).

c) $f(n) = \Theta(n^2)$ para el programa $A(n)$. **FALSO**, $f(n) = \Theta(n^3)$

```

1 def A(n):
2     atupla = tuple(range(0, n))
3     # Una tupla es una version inmutable de una lista
4     # que puede ser hasheada
5     S = set()
6     for i in range(0, n):
7         for j in range(i + 1, n):
8             # Anade la tupla (i,...,j-1) al set S
9             S.add(atupla[i:j])

```

Paso 1. La trampa. Los ciclos dan $\sum_{i=0}^{n-1} (n - 1 - i) = n(n - 1)/2$ vueltas. Si la línea 9 costara $O(1)$ el total sería $\Theta(n^2)$, pero no cuesta $O(1)$.

Paso 2. Costo de la línea 9. Hace dos cosas que dependen del largo del slice, que es $j - i$:

- $atupla[i:j]$ crea una tupla nueva y copia sus $j - i$ elementos: $\Theta(j - i)$.
- $S.add(...)$ calcula el hash de esa tupla, y el hash de una tupla se calcula recorriendo todos sus elementos: $\Theta(j - i)$. La inserción en sí es $O(1)$ esperado (todos los slices son distintos).

Paso 3. Suma total. Las líneas 2 y 5 cuestan $\Theta(n)$ y $\Theta(1)$. Para los ciclos se agrupan los pares por su diferencia $L = j - i$, que va de 1 a $n - 1$; hay exactamente $n - L$ pares con diferencia L :

$$\sum_{i < j} (j - i) = \sum_{L=1}^{n-1} L(n - L) = n \cdot n(n - 1)/2 - (n - 1)n(2n - 1)/6$$

$$= n(n - 1)[3n - (2n - 1)] / 6 = n(n - 1)(n + 1)/6 = (n^3 - n)/6$$

Entonces $f(n) = \Theta(n) + \Theta(n^2) + \Theta((n^3 - n)/6) = \Theta(n^3)$ (el $\Theta(n^2)$ es el costo fijo de cada vuelta).

Paso 4. Por qué no es $\Theta(n^2)$. Si fuera $f = O(n^2)$ existiría c con $n^3/6 \leq c \cdot n^2$ para todo n grande, es decir $n \leq 6c$, lo cual falla para $n > 6c$. Por lo tanto $f(n) \neq \Theta(n^2)$ y el enunciado es falso. (Además el set guarda todas esas tuplas, así que la memoria también es $\Theta(n^3)$.)

Paso 5. Comprobación (problema5c.py). Si f fuera $\Theta(n^2)$, duplicar n multiplicaría el tiempo por 4; con $\Theta(n^3)$, por 8. La razón medida queda muy por encima de 4 y cerca de 8, y la pendiente en log-log de las tres n más grandes es 2.91 (n^2 daría 2 y n^3 daría 3). Para n pequeñas todavía pesa el costo fijo $\Theta(n^2)$ de las vueltas:

n	llamadas a add $n(n-1)/2$	elementos copiados $(n^3-n)/6$	tiempo	$t(n) / t(n/2)$
25	300	2,600	26.20 μ s	—
50	1,225	20,825	138.81 μ s	5.30
100	4,950	166,650	1.17 ms	8.40
200	19,900	1,333,300	8.38 ms	7.18

400	79,800	10,666,600	65.54 ms	7.82
-----	--------	------------	----------	------